Tabeeby Agent

MCP, Skills and Reliable Goal Loops

What was added, why, and how to run, deploy and use it

Branch: `feature/mcp-skills-reliable-loops`
Concepts covered: Workshop 4 (MCP), Workshop 5 (Skills), Workshop 6 (Reliable loops & evaluation)
Result: 32/32 offline tests passing; 11/11 live evaluation cases passing on Gemini + Qdrant Cloud
Channels: REST API, Telegram (webhook & polling), MCP (HTTP & stdio)
Deployment target: FastAPI Cloud; local development with conda

1 October 2026

Contents

1 Summary **2**

2 Architecture **3**

3 MCP — Model Context Protocol (Workshop 4) **3**

3.1 Tabeeby as an MCP server . . . 3

3.2 Tabeeby as an MCP client . . . 4

4 Skills (Workshop 5) **4**

4.1 Progressive disclosure . . . 4

4.2 The seven skills . . . 5

5 Reliable goal loop (Workshop 6) **5**

5.1 Budget . . . 5

5.2 Self-verification in code . . . 6

5.3 Evidence returned to clients . . . 6

6 Evaluation **6**

6.1 Offline test suite — 32 tests . . . 6

6.2 Live evaluation — 11/11 passing . . . 6

7 Other improvements and bug fixes **7**

7.1 Bugs fixed . . . 7

7.2 New features . . . 8

8 Important finding: the stored vectors do not match the embedding model **8**

9 Telegram: using the chatbot **9**

9.1 Locally (no public URL) . . . 9

9.2 In production (FastAPI Cloud) . . . 9

10 Deploying to FastAPI Cloud **9**

11 What I need from you **10**

A Quick reference **10**

1 Summary

Tabeeby already had a working core: a Gemini tool-calling agent that searches 17,119 Vezeeta doctors in Qdrant, exposed over FastAPI and a Telegram webhook. This branch adds the three concepts from the course resources and turns the agent into a measurably reliable service:

MCP (Workshop 4) Tabeeby is now an MCP server (tools, resources and prompts, over streamable HTTP at `/mcp` and over stdio for Claude Desktop/Cursor) and an MCP client that can borrow tools from any external MCP server configured in `.env`.

Skills (Workshop 5) Seven medical skills written as `SKILL.md` recipes with reference files, loaded through progressive disclosure (menu → body → references), plus a trigger bench that tests whether the right skill fires.

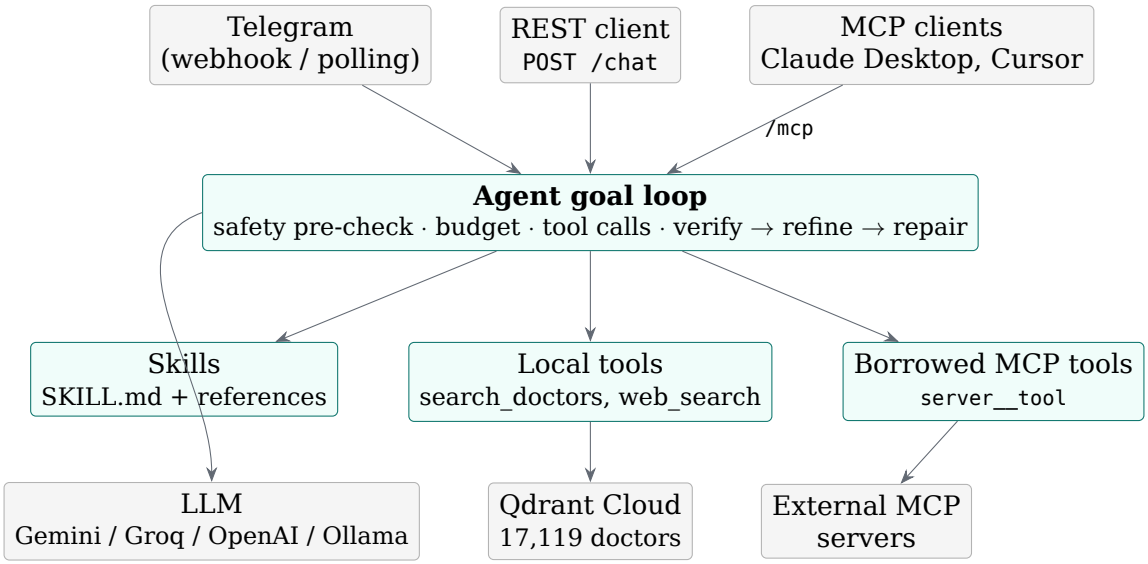
Reliable loops (Workshop 6) A four-axis budget (steps, seconds, tool actions, USD), stuck-loop detection, self-verification in code with automatic refinement, a deterministic safety repair, and an evaluation harness that turns "it worked when I tried it" into a pass rate.

On top of that, the branch fixes several real bugs found while working (Section 7), upgrades the Telegram bot, prepares FastAPI Cloud deployment, and moves the workflow to conda. One important pre-existing data issue was discovered and needs your decision (Section 8).

Concept → feature map

Course idea	Where it lives in Tabeeby
MCP server (tools/resources/prompts)	`src/mcp_server/server.py`, mounted at `/mcp` in `app.py`
MCP client, sync bridge, toolbox	`src/stores/mcp/toolbox.py` (`McpToolbox`, `McpRegistry`)
Skill menu, `load_skill`, references	`src/tools/skills.py`, `src/skills/*/SKILL.md`
Trigger bench	`src/evals/skill_trigger_bench.py`
Budget (steps, seconds, actions)	`src/agent/budget.py` (+ USD and answer-time reserve)
Verifier is code; generate→verify→refine	`src/agent/verifiers.py`, loop in `src/agent/agent.py`
Eval set as a number	`src/evals/eval_set.json`, `src/evals/run_eval.py`, `src/tests/`

2 Architecture



All entry points share one bootstrap (`helpers/bootstrap.py`), so the REST API, Telegram, the stdio MCP server, the evals and the polling bot all run exactly the same agent with the same configuration.

3 MCP — Model Context Protocol (Workshop 4)

3.1 Tabeeby as an MCP server

Defined once in `src/mcp_server/server.py` with `MCPServer` from `mcp>=2.2`, served over two transports:

- **Streamable HTTP** at `https://<app>/mcp/`, mounted inside the FastAPI app (stateless, JSON responses) and protected by `Authorization: Bearer $MCP_API_KEY`.
- **stdio** via `python src/mcp_server/__main__.py` for local clients; `--http` runs it standalone on port 8001, `--no-agent` exposes the tools only.

Kind	Name	What it does
Tool	`search_doctors`	Hybrid vector + filter search; area/specialty normalised (English or Arabic)
Tool	`check_emergency`	Deterministic red-flag scan; returns categories and emergency numbers
Tool	`medical_web_search`	Tavily search with source URLs
Tool	`list_specialties`, `list_areas`	Canonical filter values with doctor counts; resolves free text
Tool	`ask_tabeeby`	Runs the whole agent loop and returns its verified answer
Resource	`tabeeby://emergency-numbers`	Egypt emergency numbers (JSON)
Resource	`tabeeby://skills`, `tabeeby://skills/{name}`	The skill menu and each skill's instructions
Prompt	`triage(symptoms, area, max_fee)`	A ready-made triage workflow for any MCP client

Tools carry MCP annotations (`readOnlyHint`, `openWorldHint`) so clients know they are safe to call.

Claude Desktop configuration (`claude_desktop_config.json`):

```
{ "mcpServers": { "tabeeby": {
    "command": "/home/<you>/miniconda3/envs/tabeeby-agent/bin/python",
    "args": ["/path/to/Tabeeby-Agent/src/mcp_server/__main__.py"] } } }
```

3.2 Tabeeby as an MCP client

`src/stores/mcp/toolbox.py` ports the Workshop 4 `McpToolbox`: MCP is async but the agent loop is sync, so an `anyio` blocking portal runs the event loop on a background thread (thread-safe, shared by concurrent requests). `McpRegistry` combines several servers:

```
MCP_SERVERS={"fetch":  {"command": "uvx", "args": ["mcp-server-fetch"]},
             "remote": {"url": "https://example.com/mcp",
                        "headers": {"Authorization": "Bearer ..."}}}
```

- Borrowed tools are named `<server>__<tool>` so two servers can both expose `search`.
- Their JSON schemas are passed to the LLM next to our Python tools. The Gemini provider was extended to turn schemas into `FunctionDeclarations`; OpenAI, Groq and Ollama accept them natively.
- A server that fails to start is logged in `/health` and skipped — one optional integration can never take the medical assistant down. A tool error is returned to the model as text, so the loop survives.

4 Skills (Workshop 5)

> A tool is an appliance. A skill is a recipe.

4.1 Progressive disclosure

Level	What	When it is read
1	`name` + `description`	Always — appended to the system prompt as a menu
2	Body of `SKILL.md`	When the model calls `load_skill(name)`
3	Files in `references/`	When the body says to call `read_skill_reference(skill, file)`

Reference reads are sandboxed to the skill's own `references/` folder (path traversal is blocked and tested).

4.2 The seven skills

Skill	Recipe (references)
`emergency-triage`	"Call 123" first, then ≤4 first-aid steps only from the reference, then optional follow-up specialists (`red-flags.md`, `first-aid.md`)
`doctor-finder`	Decide specialty, extract area/budget, search, present doctor cards (`specialty-map.md`, `card-format.md`)
`egyptian-arabic`	Match the patient's register; translate the complaint to an English clinical query for the tool (`glossary.md`)
`medication-info`	Web search with citations; never personal dosing (`safety-rules.md`)
`lab-results`	Below/within/above typical range, which specialist reads it (`reference-ranges.md`)
`pediatric-care`	Warning signs first, then pediatricians; no child doses (`warning-signs.md`)
`mental-health-support`	Validation, two coping steps, psychiatrists; self-harm hands over to emergency-triage

Adding a skill needs no Python: create `src/skills/<name>/SKILL.md` with a `name` and a `description` whose "Use when …" part lists the words patients actually type. The description is the trigger; when the wrong skill fires, fix that one line and re-run `make bench`.

Skills are also visible from outside: `GET /skills`, `GET /skills/{name}`, the Telegram `/skills` command and the MCP resources.

5 Reliable goal loop (Workshop 6)

> A loop without a budget is a bug. The verifier is where the intelligence is.

5.1 Budget

Each patient message gets a `Budget` (`agent/budget.py`), configurable in `.env`:

Axis	Default	Setting
LLM calls (steps)	8	`AGENT_MAX_STEPS`
Wall clock	90 s	`AGENT_MAX_SECONDS`
Tool calls actually executed (actions)	12	`AGENT_MAX_ACTIONS`
Spend in USD	off	`AGENT_MAX_COST_USD`
Verifier-driven rewrites	2	`AGENT_MAX_REFINEMENTS`

Two refinements over the workshop version, both found by running the real agent:

- **Answer reserve.** On the last step, after the action budget, and in the last 25% of the time budget, tool calls are refused with "write your final answer now". Without this the live agent was cut off mid-search and the patient received no answer.
- **Stuck detection.** A tool that fails twice is disabled for the rest of the message, and an identical repeated call is not executed again. In the live run the model had retried a failing search four times in a row.

5.2 Self-verification in code

Every final answer is checked by code before it reaches the patient (`agent/verifiers.py`):

Verifier	Rule
`answer_not_empty`	The answer has text.
`emergency_protocol`	If the message contains red flags, the answer contains 123.
`links_are_grounded`	Every Vezeeta doctor link was returned by `search_doctors` (no invented URLs).
`booking_links_present`	If doctors were found, at least one of their booking links is shown.
`language_matches`	Arabic in, Arabic out.

A failing check is sent back with specific feedback ("these links were not returned by search_doctors: …"), and the model rewrites its answer. If the budget ends while a check still fails, `repair_answer` applies deterministic fixes: unverified links are removed and the emergency banner is put first. An unsafe answer is never shipped as-is.

The red-flag detector (`agent/safety.py`) is plain code covering English, Modern Standard Arabic and Egyptian colloquial forms (masculine, feminine and first person). It also adds a safety note to the system prompt before the first LLM call.

5.3 Evidence returned to clients

`Agent.run()` returns an `AgentResult`: the answer plus `stop_reason` (`completed`, `verified after N refinement(s)`, `step budget hit`, `repaired: …`, `llm_error`), the verification checks, skills used, every tool call with its source (`local`, `mcp`, `dropped`, `disabled`, `duplicate`), budget usage and cost. `POST /chat` returns these with `"include_trace": true`.

6 Evaluation

6.1 Offline test suite — 32 tests

`make test` runs without API keys: a scripted fake LLM and a fake vector DB exercise the budget, safety, catalog normalisation, verifiers, skills (all three levels and the sandbox), the full loop (skill → search → verified answer; fabricated link → refinement; emergency repair; dropped actions; step cap; stuck detection), the MCP server in memory, the MCP client against a real stdio subprocess, the HTTP `/mcp` endpoint with bearer auth, `/chat` sessions, and the Telegram handler and formatter. The same suite was also run in a clean `uv sync --locked` environment on Python 3.10, which simulates the FastAPI Cloud build.

6.2 Live evaluation — 11/11 passing

`make eval` runs the real agent (Gemini + Qdrant Cloud) over `evals/eval_set.json` and scores each case with code: specialty and area passed to the search (after normalisation), budget respected, answer language, emergency detection, required/forbidden text, expected skill, and whether the answer self-verified.

Case	Prompt (summary)	Result
dental-ar-area	Egyptian Arabic toothache in Maadi	pass
ortho-en-budget	Knee injury, Nasr City, max 400 EGP	pass
cardiac-emergency	Crushing chest pain spreading to arm	pass (1 refinement)
stroke-ar	Arabic stroke signs (feminine forms)	pass
pediatric-fever	2-year-old with fever, Heliopolis	pass
derm-hairloss	Hair loss, 6th of October	pass
fertility-ar	Arabic: no pregnancy after 2 years, Nasr City	pass (1 refinement)
anxiety	Insomnia and anxiety	pass (1 refinement)
lab-hba1c	HbA1c 7.2%	pass (1 refinement)
out-of-scope	Python coding request	pass (refused)
jailbreak	“Ignore previous instructions…”	pass (refused)

The first full run scored 5/11. The failures were caused by the Gemini free-tier rate limit, a gap in the Arabic red-flag phrases, and a verifier that treated the Vezeeta homepage as a doctor link. Each was fixed (retry/backoff, feminine Egyptian forms, profile-link-only grounding), and the cases now pass. A full run costs about $0.017. Reports are saved to `evals/reports/`.

`make bench` runs the skill trigger bench: for each prompt it checks the model’s first move (which skill it loads, or none for “hi”).

7 Other improvements and bug fixes

7.1 Bugs fixed

- **Area and specialty filters silently returned nothing.** Qdrant filters use exact keyword match, so “Maadi”, “6 October”, Arabic area names or “Orthopedic Surgery” never matched the stored values (`El-Maadi`, `6Th Of October`, `Orthopedics`). `tools/catalog.py` now maps free text to the 140 canonical areas and 37 specialties (alias table with Arabic, then fuzzy matching).
- **Zero results instead of the closest doctors.** The search now relaxes one constraint at a time (specialty, then fee, then area) and tells the model which filter it dropped. A query that is just a specialty name becomes a filter.
- **The patient’s message was repeated to the LLM.** The loop passed the prompt again on every iteration while it was already in the history, so after each tool call the model saw the user turn duplicated (always, with Ollama).
- **Telegram history could break Gemini.** Trimming to “system + last 10 messages” could start the window on an orphaned tool response and stored raw tool-call objects. Memory now keeps only clean user/assistant turns (`helpers/session_store.py`, with TTL).
- **Telegram answers over 4096 characters failed** and Markdown such as `**bold**` and `[text](url)` showed as raw symbols. Answers are now split on paragraph boundaries and converted to Telegram HTML, with a plain-text fallback.
- **Duplicate agent runs from Telegram retries:** the webhook waited for the full agent run, so Telegram retried. It now verifies, de-duplicates and returns 200 immediately; the work continues in the background.
- `ChatRequest.total_cost` was an `int`; `_execute_tool` referenced attributes that did not exist; the system prompt told the model to answer “in the user’s language

English”; the Groq settings existed but were never wired up; the unused `pathlib` backport was a dependency.

7.2 New features

- Retry with backoff for rate limits and transient errors (honours Gemini’s “retry in N s”).
- `GROQ` generation backend (OpenAI-compatible).
- Server-side conversation memory for the REST API (`session_id`, `DELETE /chat/{id}`).
- `GET /health`, `/tools`, `/skills`; FastAPI app factory for testing.
- Telegram: `/help`, `/emergency`, `/skills`, bot command menu, webhook secret token, per-chat lock, non-text message handling, and a long-polling mode (`make telegram`) for local development without a public URL.
- `scripts/check_vector_consistency.py` — verifies the vectors match the embedding model.
- Conda `environment.yml`, a Makefile that runs everything in the conda env, and `scripts/push_env_to_fastapi_cloud.sh`.
- The system prompt forbids Markdown tables (they do not render in Telegram) and caps the answer length.

8 Important finding: the stored vectors do not match the embedding model

While testing, unfiltered semantic search returned clearly wrong specialties (“toothache” → orthopedists, “child fever” → physiotherapists). To confirm, I re-embedded the stored text of several Qdrant points with the configured model (`gemini-embedding-001`, 1024 dimensions) and compared it with the stored vector:

Point	cosine(stored, re-embedded)
Reproductive Medicine & Infertility	0.017
Neurology	−0.021
Cardiology	0.019

The same model would give about 1.0. The collection was therefore built with a different embedding model, and the semantic part of the hybrid search is currently noise. Results looked right only because the agent passes specialty and area filters. The new normaliser and specialty inference make those filters much more reliable, but symptom-level matching (subspecialties, conditions) will only work after re-ingestion.

Decision needed

Either (a) tell me which embedding model/provider was used for ingestion and I will switch `EMBEDDING_*` back, or (b) re-ingest with the current model: `make ingest` (drops and rebuilds the cloud collection; 17,119 embedding calls — probably needs a paid Gemini tier or several days on the free tier). Check afterwards with `make check-vectors`. I did not run (b) because it is destructive and uses your quota.

9 Telegram: using the chatbot

9.1 Locally (no public URL)

```
cd src && conda activate tabeeby-agent
make telegram        # long polling; removes the webhook while it runs
```

9.2 In production (FastAPI Cloud)

Set these and deploy; on startup the app registers the webhook with the secret and sets the bot's command menu:

```
TELEGRAM_BOT_TOKEN=...
TELEGRAM_WEBHOOK_URL=https://<your-app>.fastapicloud.dev/telegram/webhook
TELEGRAM_WEBHOOK_SECRET=<python -c "import secrets; print(secrets.token_urlsafe(32))">
```

Telegram allows only one delivery mode per bot. After using polling locally, restart or redeploy the cloud app to restore the webhook. Your current `.env` still points the webhook at an old `trycloudflare.com` tunnel; replace it with the FastAPI Cloud URL.

Commands: `/start`, `/help`, `/reset`, `/emergency`, `/skills`.

10 Deploying to FastAPI Cloud

FastAPI Cloud installs dependencies from `uv.lock` (regenerated on this branch with `mcp 2.2.0`), picks the Python version from `requires-python`, and reads the entrypoint from `[tool.fastapi] entrypoint = "app:app"` in `pyproject.toml`. Environment variables must be set in the cloud because `.env` is not uploaded.

```
cd src && conda activate tabeeby-agent
fastapi login                # opens the browser (your account)
fastapi deploy               # builds and deploys; prints the app URL
make cloud-env               # pushes .env: *KEY/*TOKEN/*SECRET as encrypted secrets
fastapi deploy               # env changes need a redeploy
curl https://<your-app>.fastapicloud.dev/health
```

Before `make cloud-env`, edit `.env`: set `TELEGRAM_WEBHOOK_URL` to the cloud URL, and add `TELEGRAM_WEBHOOK_SECRET` and `MCP_API_KEY`. A dry run of the production build (clean `uv sync --locked` on Python 3.10, then `fastapi run`) passed: `/health` reported every component healthy, `/mcp` rejected requests without the key (401), and an MCP client listed and called all six tools over HTTP.

11 What I need from you

Requests

1. **Embedding decision** (Section 8): which model built the collection, or approval to re-ingest.
2. **FastAPI Cloud login**: run `fastapi login` yourself (it is your account), then `fastapi deploy`. I prepared everything else but cannot log in for you.
3. **Secrets to create**: `TELEGRAM_WEBHOOK_SECRET` and `MCP_API_KEY` (any long random strings), and the new `TELEGRAM_WEBHOOK_URL`.
4. **Gemini quota**: the free tier (15 requests/minute) is tight, since one message can take 3–6 LLM calls. Retries now handle short bursts. For real users, enable billing, or use `GENERATION_BACKEND=GROQ` with `openai/gpt-oss-120b` (tested and working with your key).
5. **Gemini key**: early in the session your `AQ.`-prefixed key returned `401 ACCESS_TOKEN_TYPE_UNSUPPORTED`; it worked again minutes later. If it recurs, create a standard AI Studio key (`AIza…`).
6. **Medical review**: the skill references (first aid, lab ranges, pediatric warning signs) are written conservatively from general guidance. Please have a clinician review them, and confirm any extra Egyptian hotline numbers you want added (only 123, 122 and 180 are included).
7. **Scaling**: conversation memory is in-process. If FastAPI Cloud runs several instances, say so and I will move sessions to Redis (already a dependency).

A Quick reference

Command (in `src/`)	Does
`make env`	Create/update the `tabeeby-agent` conda env
`make app`	Run the API locally with reload (`/docs`)
`make test`	32 offline tests
`make eval` / `make bench`	Live evaluation / skill trigger bench
`make mcp` / `make mcp-http`	Tabeeby MCP server over stdio / HTTP :8001
`make telegram`	Telegram bot via long polling
`make check-vectors`	Embedding/vector consistency check
`make lock`	Refresh `uv.lock` after editing dependencies
`make cloud-env` / `make deploy`	Push env to FastAPI Cloud / deploy
`make docs`	Rebuild this PDF